

LAB-08

Refer the application of Book management as described in Lab-6. Create the same application with the help of entity framework so that the data is stored in the database. First, write the Book model, configure context class, connection string, configuration in the program.cs file and then perform migration. After the database is created through the code, write Book controller and views with appropriate CRUD operations.

[Book.cs](#)

```
using System.ComponentModel.DataAnnotations;
```

```
namespace Book_Management.Models
{
    public class Book
    {
        [Key]
        public int BookId { get; set; }

        [Required]
        public string Title { get; set; }

        [Required]
        public string Author { get; set; }

        public string Category { get; set; }

        [Range(0, double.MaxValue)]
        public decimal Price { get; set; }

        public int PublishedYear { get; set; }
    }
}
```

BookRepository.cshtml

```
using Book_Management.Data;
using Book_Management.Models;
using Microsoft.EntityFrameworkCore;

namespace Book_Management.Repositories
{
    public class BookRepository : IBookRepository
    {
        private readonly ApplicationDbContext _context;

        public BookRepository(ApplicationDbContext context)
        {
            _context = context;
        }

        public async Task<IEnumerable<Book>> GetAll()
        {
            return await _context.Books.ToListAsync();
        }

        public async Task<Book?> GetById(int id)
        {
            return await _context.Books.FindAsync(id);
        }

        public async Task Add(Book book)
        {
            await _context.Books.AddAsync(book);

            await _context.SaveChangesAsync();
        }

        public async Task Update(Book book)
        {
            _context.Books.Update(book);

            await _context.SaveChangesAsync();
        }
    }
}
```

```

public async Task Delete(int id)
{
    var book = await _context.Books.FindAsync(id);

    if (book != null)
    {
        _context.Books.Remove(book);

        await _context.SaveChangesAsync();
    }
}
}

```

IBookRepository.cshtml

```

using Book_Management.Models;

namespace Book_Management.Repositories
{
    public interface IBookRepository
    {
        Task<IEnumerable<Book>> GetAll();

        Task<Book?> GetById(int id);

        Task Add(Book book);

        Task Update(Book book);

        Task Delete(int id);
    }
}

```

ApplicationContext.cs

```
using Microsoft.EntityFrameworkCore;
using Book_Management.Models;
namespace Book_Management.Data
{
    public class ApplicationDbContext:DbContext
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) :
base(options) { }
        public DbSet<Book> Books { get; set; }
    }
}
```

BookController.cshtml

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using Book_Management.Data;
using Book_Management.Models;

namespace Book_Management.Controllers
{
    public class BooksController : Controller
    {
        private readonly ApplicationDbContext _context;

        public BooksController(ApplicationDbContext context)
        {
            _context = context;
        }

        // GET: Books
        public async Task<IActionResult> Index()
        {
            var books = await _context.Books.ToListAsync();

            return View(books);
        }
    }
}
```

```

// GET: Books/Details/5
public async Task<IActionResult> Details(int? id)
{
    if (id == null)
    {
        return NotFound();
    }

    var book = await _context.Books
        .FirstOrDefaultAsync(b => b.BookId == id);

    if (book == null)
    {
        return NotFound();
    }

    return View(book);
}

// GET: Books/Create
public IActionResult Create()
{
    return View();
}

// POST: Books/Create
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Create(Book book)
{
    if (ModelState.IsValid)
    {
        _context.Add(book);

        await _context.SaveChangesAsync();

        return RedirectToAction(nameof(Index));
    }
}

```

```

        return View(book);
    }

    // GET: Books/Edit/5
    public async Task<IActionResult> Edit(int? id)
    {
        if (id == null)
        {
            return NotFound();
        }

        var book = await _context.Books.FindAsync(id);

        if (book == null)
        {
            return NotFound();
        }

        return View(book);
    }

    // POST: Books/Edit/5
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Edit(int id, Book book)
    {
        if (id != book.BookId)
        {
            return NotFound();
        }

        if (ModelState.IsValid)
        {
            try
            {
                _context.Update(book);

                await _context.SaveChangesAsync();
            }
            catch (DbUpdateConcurrencyException)

```

```

        {
            if (!_context.Books.Any(e => e.BookId == book.BookId))
            {
                return NotFound();
            }

            throw;
        }

        return RedirectToAction(nameof(Index));
    }

    return View(book);
}

```

```

// GET: Books/Delete/5
public async Task<IActionResult> Delete(int? id)
{
    if (id == null)
    {
        return NotFound();
    }

    var book = await _context.Books
        .FirstOrDefaultAsync(b => b.BookId == id);

    if (book == null)
    {
        return NotFound();
    }

    return View(book);
}

```

```

// POST: Books/Delete/5
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> DeleteConfirmed(int id)
{
    var book = await _context.Books.FindAsync(id);
}

```

```

        if (book != null)
        {
            _context.Books.Remove(book);

            await _context.SaveChangesAsync();
        }

        return RedirectToAction(nameof(Index));
    }
}

```

Index.cshtml

```
@model IEnumerable<Book_Management.Models.Book>
```

```
@{
    ViewData["Title"] = "Books";
}
```

```
<h1>Book List</h1>
```

```
<p>
    <a asp-action="Create" class="btn btn-primary">
        Add New Book
    </a>
</p>
```

```
<table class="table table-bordered">
    <thead>
        <tr>
            <th>Book ID</th>
            <th>Title</th>
            <th>Author</th>
            <th>Category</th>
            <th>Price</th>
            <th>Published Year</th>
            <th>Actions</th>

```

```
        </tr>
    </thead>

    <tbody>
        @foreach (var book in Model)
        {
            <tr>
                <td>@book.BookId</td>
                <td>@book.Title</td>
                <td>@book.Author</td>
                <td>@book.Category</td>
                <td>@book.Price</td>
                <td>@book.PublishedYear</td>

                <td>
                    <a asp-action="Details"
                       asp-route-id="@book.BookId"
                       class="btn btn-info">
                        Details
                    </a>

                    <a asp-action="Edit"
                       asp-route-id="@book.BookId"
                       class="btn btn-warning">
                        Edit
                    </a>

                    <a asp-action="Delete"
                       asp-route-id="@book.BookId"
                       class="btn btn-danger">
                        Delete
                    </a>
                </td>
            </tr>
        }
    </tbody>
</table>
```

Create.cshtml

@model Book_Management.Models.Book

<h1>Add New Book</h1>

<form asp-action="Create">

<div class="mb-3">

<label asp-for="Title"></label>

<input asp-for="Title" class="form-control" />

</div>

<div class="mb-3">

<label asp-for="Author"></label>

<input asp-for="Author" class="form-control" />

</div>

<div class="mb-3">

<label asp-for="Category"></label>

<input asp-for="Category" class="form-control" />

</div>

<div class="mb-3">

<label asp-for="Price"></label>

<input asp-for="Price" class="form-control" />

</div>

<div class="mb-3">

<label asp-for="PublishedYear"></label>

<input asp-for="PublishedYear" class="form-control" />

</div>

<button type="submit" class="btn btn-primary">

Save

</button>

<a asp-action="Index" class="btn btn-secondary">

Back

</form>

Details.cshtml

@model Book_Management.Models.Book

<h1>Book Details</h1>

<hr />

<dl class="row">

<dt class="col-sm-3">Book ID</dt>

<dd class="col-sm-9">@Model.BookId</dd>

<dt class="col-sm-3">Title</dt>

<dd class="col-sm-9">@Model.Title</dd>

<dt class="col-sm-3">Author</dt>

<dd class="col-sm-9">@Model.Author</dd>

<dt class="col-sm-3">Category</dt>

<dd class="col-sm-9">@Model.Category</dd>

<dt class="col-sm-3">Price</dt>

<dd class="col-sm-9">@Model.Price</dd>

<dt class="col-sm-3">Published Year</dt>

<dd class="col-sm-9">@Model.PublishedYear</dd>

</dl>

<a asp-action="Edit"

asp-route-id="@Model.BookId"

class="btn btn-warning">

Edit


```
<a asp-action="Index" class="btn btn-secondary">
  Back
</a>
```

Edit.cshtml

@model Book_Management.Models.Book

```
<h1>Edit Book</h1>
```

```
<form asp-action="Edit">
```

```
  <input type="hidden" asp-for="BookId" />
```

```
  <div class="mb-3">
```

```
    <label asp-for="Title"></label>
```

```
    <input asp-for="Title" class="form-control" />
```

```
    <span asp-validation-for="Title" class="text-danger"></span>
```

```
  </div>
```

```
  <div class="mb-3">
```

```
    <label asp-for="Author"></label>
```

```
    <input asp-for="Author" class="form-control" />
```

```
    <span asp-validation-for="Author" class="text-danger"></span>
```

```
  </div>
```

```
  <div class="mb-3">
```

```
    <label asp-for="Category"></label>
```

```
    <input asp-for="Category" class="form-control" />
```

```
  </div>
```

```
  <div class="mb-3">
```

```
    <label asp-for="Price"></label>
```

```
    <input asp-for="Price" class="form-control" />
```

```
  </div>
```

```
  <div class="mb-3">
```

```
    <label asp-for="PublishedYear"></label>
```

```
    <input asp-for="PublishedYear" class="form-control" />
```

</div>

<button type="submit" class="btn btn-primary">
Update
</button>

<a asp-action="Index" class="btn btn-secondary">
Back

</form>

Delete.cshtml

@model Book_Management.Models.Book

<h1>Delete Book</h1>

<h3>Are you sure you want to delete this book?</h3>

<hr />

<div>

<p>
 Title:
 @Model.Title
</p>

<p>
 Author:
 @Model.Author
</p>

<p>
 Category:
 @Model.Category
</p>

<p>

```

        <strong>Price:</strong>
        @Model.Price
    </p>

    <form asp-action="Delete">

        <input type="hidden" asp-for="BookId" />

        <button type="submit" class="btn btn-danger">
            Delete
        </button>

        <a asp-action="Index" class="btn btn-secondary">
            Cancel
        </a>

    </form>

</div>

```

Appsettings.json

```

{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=(localdb)\MSSQLLocalDB;Database=BookManagementDB;Trusted_Connection=True;TrustServerCertificate=True;"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}

```

Program.cs

```
using Book_Management.Data;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    // The default HSTS value is 30 days. You may want to change this for production
    scenarios, see https://aka.ms/aspnetcore-hsts.
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseRouting();

app.UseAuthorization();

app.MapStaticAssets();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Books}/{action=Index}/{id?}")
    .WithStaticAssets();

app.Run();
```

